

Laborator 8 – Utilizarea coprocesorului matematic

8.1 Scopul lucrării

Lucrarea de față își propune familiarizarea utilizatorului cu folosirea funcțiilor coprocesorului matematic, utilizarea instrucțiunilor de lucru cu numere reale și a altor funcții ale coprocesorului.

8.2 Considerații generale

Cu toate că procesorul 8086 respectiv 80286, 80386 și 80486 au o serie de instrucțiuni aritmetice puternice (care există în repertoriul microprocesoarelor de generația a doua, cum sunt înmulțirea și împărțirea binară) nu sprijină operațiile aritmetice cu numerele reprezentate în virgula flotantă sau pentru numerele întregi reprezentate pe mai mulți octeți. Dacă vrem să efectuăm astfel de operații atunci aceste operații trebuie să fie realizate prin biblioteci de macro-uri sau prin subrutine. O cale mai simplă este utilizarea limbajelor de nivel înalt, în aceste limbaje operațiile de mai sus fiind realizate prin biblioteci.

Pentru evitarea greutăților amintite firma INTEL a dezvoltat coprocesorul aritmetic INTEL 8087 (80287, 80387). Acestea după cum arată și numele sunt “co”-procesoare, care cooperează cu procesorul principal al calculatorului. Coprocesorul nu poate lucra independent de procesor. Coprocesorul nu poate să extragă singur instrucțiunile din memorie, acesta fiind realizat de procesor.

8.2.1 Principiul de funcționare

Coprocesorul se inițializează odată cu activarea semnalului RESET, general în sistemul de calcul. În urma acestui semnal coprocesorul este adus într-o stare inițială (cu mascarea erorilor, ștergerea registrelor, inițializarea stivei, rotunjirea implicită, etc.). După prima instrucțiune executată de procesor coprocesorul poate detecta cu ce procesor lucrează (dacă este 8086 pe pinul 34 va fi 0 logic-semnalul / BHE-iar dacă este 8088 va fi 1 logic-semnalul/SS0). În funcție de tipul procesorului, coprocesorul se configurează corespunzător.

Coprocesorul se conectează pe magistrala locală a procesorului prin liniile de adrese/date, stare, ceas, ready, reset, test și request/grant. Fiind conectat la magistrala locală microprocesorul permite coprocesorului accesul la toată memoria și resursele de intrare/ieșire, prin intermediul cererii de magistrală request/grant.

Cele două procesoare lucrând în paralel este nevoie de sincronizarea proceselor care rulează în ele.

De regulă sincronizarea erorilor și a instrucțiunilor este rezolvată de compilatoare sau asamblatoare, iar sincronizarea datelor trebuie să o facă utilizatorul care lucrează în limbajul de asamblare.

În interiorul coprocesorului avem o memorie de 80 de octeți organizată sub forma unei stive de opt elemente de 10 octeți fiecare. Pe cei zece octeți numerele în virgulă mobilă se reprezintă sub format intern, cu precizie extinsă. Coprocesorul poate accesa memoria calculatorului, cu orice mod de adresare cunoscut de 8086, orice dată de format legal. Datele aduse din memorie se convertesc în formatul intern al coprocesorului și se pun la vârful stivei. La scrierea în memoria principală se face conversia din formatul intern în formatul specificat de utilizator.

Condiția pentru efectuarea operațiilor în virgulă flotantă în coprocesor este ca operandul (pentru operații cu doi operanzi, cel puțin unul din operanzi) să fie în vârful stivei. Deci cu ajutorul coprocesorului putem efectua următoarele operații:

- citirea datelor în memoria internă a coprocesorului (pe stivă) din memoria calculatorului
- efectuarea operațiilor aritmetice necesare
- scrierea rezultatului în memoria calculatorului.

8.2.2 Tipuri de date cunoscute de INTEL 8087

Marele avantaj al coprocesorului este faptul că nu lucrează doar cu numere în virgulă flotantă, ci și cu numere întregi și recunoaște și tipuri de date zecimale împachetate. Deci dacă avem o operație complicată de efectuat între numere întregi și ea trebuie să fie foarte rapidă și acest lucru de poate realiza cu ajutorul coprocesorului, nu avem nevoie de conversie, costisitoare în timp, din întreg în virgulă flotantă și după aceea invers doar pentru ca coprocesorul să poată lucra cu ele.

Tipurile de date în virgulă flotantă sunt:

- Real scurt, reprezentat pe 32 bit (1 bit de semn, 23 bit mantisa, 8 bit caracteristica)
- Real lung, reprezentat pe 64 bit (1 bit de semn, 52 bit mantisa, 11 bit caracteristica)
- Real cu precizie ridicată, reprezentat pe 80 bit (1 bit de semn, 64 bit mantisa, 15 bit caracteristica)

8.2.3 Erori de operație (excepții)

La efectuarea operațiilor în virgulă flotantă putem avea numeroase erori începând de la erori logaritmice triviale, până la erorile provenite din limitele reprezentării. Acestea le vom numi excepții. În continuare vom cunoaște tipurile acestor erori și posibilitățile principale de manevrare a lor.

În cazul apariției erorilor, coprocesorul poate avea două tipuri de comportare. Anunță eroarea printr-o întrerupere dacă utilizatorul validează acest lucru. Dacă nu validăm întreruperea, coprocesorul va trata intern eroarea și în funcție de erorile apărute va acționa în modul prezentat în continuare. Proiectanții coprocesorului au clasificat erorile în următoarele 6 clase:

Invalid operation: operație invalidă

Aceasta poate fi: depășire superioară sau inferioară a stivei interioare a coprocesorului. Depășirea inferioară apare în cazul în care vrem să accesăm un element din stivă care nu există. Acestea sunt de obicei erori (destul de grave) algoritmice; coprocesorul nu afectează operația. Avem rezultat nedefinit în cazul în care încercăm să împărțim 0. 0 cu 0. 0, coprocesorul nu este pregătit pentru aceasta. Cazuri similare apar la scăderea lui infinit din infinit, etc. Aceste erori (deși se pot evita prin algoritm) nu sunt erori atât de grave ca cele de depășire inferioară sau superioară a stivei. Avem tot acest “rezultat” dacă o funcție de coprocesor este apelată ca parametrii necorespunzători. În cazul apariției rezultatului nedefinit coprocesorul înscrie în caracteristică o valoare rezervată pentru acest caz (biți de zero).

Overflow: depășire superioară

Rezultatul depășește numărul cel mai mare ce se poate reprezenta. Coprocesorul înscrie infinit în locul rezultatului și continuă lucrul.

Zero Divisor: împărțire cu zero

Împărțitorul împărțirii de efectuat este zero, iar deîmpărțitul nu este zero sau infinit. Coprocesorul înscrie în locul rezultatului infinit și continuă lucrul.

Underflow: depășire inferioară

Valoarea rezultatului în modul este mai mică decât numărul cel mai mic reprezentabil. Rezultatul va fi zero, coprocesorul continuă lucrul.

Denormalized: Operand operand nenormalizat

Această excepție apare dacă unul din operanzi nu este normalizat sau rezultatul nu se poate reprezenta normalizat (de exemplu este atât de mic încât este imposibilă normalizarea lui). Coprocesorul continuă lucrul (valorile diferite de zero se pierd, vor fi zero).

Inexact result: Rezultat inexact

Rezultatul operației este inexact din cauza unor rotunjiri prescrise sau necesare. Putem avea astfel de rezultate după împărțire, dacă împărțim de exemplu 2. 0 cu 3. 0 rezultatul se poate reprezenta doar ca o fracție infinită. Coprocesorul efectuează rotunjirea și continuă lucrul.

Cele de mai sus sunt prezentate în funcție de gravitatea erorii. Dacă apare o depășire de stivă atunci programul este eronat; nu merită să se continue programul.

În același timp nu e nevoie să se trateze o eroare de rotunjire. Nici pe hârtie nu putem manevra ușor fracții cu repetiție infinită sau cu numere iraționale. Din punct de vedere practic este indiferent dacă pierdem a 20-a cifră a fracției sau nu, deoarece nu aceasta este elementul care poartă informația de bază. Pentru rezolvarea acestei probleme este necesară o analiză adâncă a situațiilor și rezultatelor care pot apărea în timpul execuției, a preciziei de reprezentare a numerelor, timpul de rulare și mărimea memoriei. După cum am văzut la reprezentarea numerelor, precizia numerelor reale scurte nu este de ajuns pentru orice aplicație practică. Precizia numerelor reale lungi este mai mult ca sigur suficientă, dar necesită un spațiu dublu de memorie.

8.3 Setul de instrucțiuni al coprocesorului

Programarea coprocesorului se face în limbajul de asamblare cu ajutorul instrucțiunii ESC. Această instrucțiune trimite pe magistrala de date un cod de operație pe 6 biți și dacă este necesar trimite pe magistrala de date o adresă de memorie. Coprocesorul sesizează și “captează” instrucțiunea ce i se adresează și începe execuția instrucțiunii. Există două posibilități de resincronizare a procesorului cu coprocesorul, ambele la inițiativa procesorului:

- procesorul testează starea coprocesorului;
- procesorul lansează o instrucțiune WAIT;

8.3.1 Instrucțiuni de transfer de date

Instrucțiunile de transfer de date asigură schimbul de date între memoria calculatorului și stiva coprocesorului. Ele se pot împărți în următoarele categorii:

Instrucțiuni de încărcare (LOAD)

FILD adr Încarcă pe stivă variabila întreagă de la adresa “adr”. Variabila din memorie de tipul definit la declararea lui (DB, DW, DD) se convertește în formatul intern al coprocesorului în timpul încărcării

FLD adr Încarcă pe stivă valoarea reală (scurtă sau lungă) de la adresa de memorie “adr”. Variabila din memorie de tipul definit la declararea lui (DD, DQ, DT) se convertește în formatul intern al coprocesorului în timpul încărcării.

FBLD adr Încarcă pe stivă variabila din memorie de tipul zecimal împachetat (definit cu DT) de la adresa de memorie “adr”. Are loc convertirea în formatul intern al coprocesorului în timpul încărcării.

Instrucțiuni de memorare (STORE)

FIST adr Memorează la adresa “adr” valoarea de pe stivă (ST (0)) ca număr. Valoarea memorată poate fi numai întreg pe cuvânt sau întreg scurt, în funcție de definiția de la adresa “adr” (DW sau DD). indicatorul de stivă nu se modifică în urma memorării. În timpul memorării are loc convertirea.

FISTP adr Memorează la adresa “adr” valoarea de pe stivă (ST (0)) ca număr întreg. Valoarea memorată poate fi orice număr întreg (pe cuvânt, scurt sau lung în funcție de definiția de la adresa “adr” DW, DD sau DQ). În timpul memorării are loc convertirea necesară. Instrucțiunea afectează stiva: ST (0) este eliminat prin decrementarea indicatorului de stivă.

FST adr Memorează la adresa “adr” valoarea de pe stivă (ST (0)) ca număr întreg. Valoarea memorată poate fi număr întreg scurt sau dublă precizie în funcție de definiția de la adresa “adr” DD sau DQ. În timpul memorării are loc convertirea necesară. Indicatorul de stivă și conținutul stivei nu se modifică în urma memorării.

FSTP adr Memorează la adresa “adr” valoarea de pe stivă (ST (0)) ca număr în reprezentarea în virgulă mobilă. Valoarea memorată poate număr real scurt, cu precizie dublă sau extins, în funcție de definiția de la adresa “adr” (DD, DQ sau DT). În timpul memorării are loc convertirea necesară din formatul intern. Instrucțiunea afectează stiva: ST (0) este eliminat prin decrementarea indicatorului de stivă.

FBSTP adr Memorează la adresa “adr” valoarea de pe stivă (ST (0)) ca număr zecimal împachetat (definit la “adr” de regulă cu DT). Indicatorul de stivă este decrementat. În timpul memorării are loc convertirea necesară din formatul intern.

8.3.2 Instrucțiuni transfer de date intern

FLD ST (i) Pune pe stivă valoarea de pe ST (i). Deci valoarea din ST (i) se va găsi de două ori: în ST (0) și ST (i+1).

FST ST (i) Valoarea din ST (0) este copiată în elementul “i” din stivă. Valoarea veche din ST (i) se pierde.

FSTP ST (i) Valoarea ST (0) este copiată în elementul “i” din stivă. Valoarea veche din ST (i) se pierde. ST (0) este eliminat prin decrementarea indicatorului din stivă.

FXCH ST (i) Se schimbă între ele ST (0) și ST (i).

8.3.3 Instrucțiuni încărcare a constantelor

FLDZ	Încarcă zero pe vârful stivei.
FLD1	Încarcă 1. 0 pe vârful stivei.
FLDPI	Încarcă “pi” pe vârful stivei
FLDL2T	Încarcă pe vârful stivei log (10)
FLDL2E	Încarcă pe vârful stivei log (e)
FLDLG2	Încarcă pe vârful stivei log (2)
FLDLN2	Încarcă pe vârful stivei ln (2)

8.3.4 Instrucțiuni aritmetice și de comparare

Instrucțiunile aritmetice sunt în general cu doi operanzi. Unul din operanzi este totdeauna în vârful stivei și de regulă tot aici se generează rezultatul. Operațiile de bază se pot executa fără restricții cu următoarele variante:

-se scrie numai mnemonica instrucțiunii fără operand. În acest caz operanzii implicați sunt ST (0) și ST (1).

-se scrie mnemonica instrucțiunii și operandul. Operandul poate fi o locație de memorie sau un element de pe stivă (bineînțeles că și ST (1) se poate dar e inutil).

-se scrie mnemonica instrucțiunii și doi operanzi: primul un element de pe stivă (nu ST (0), al doilea ST (0). În acest caz rezultatul se va depune în locul primului operand iar ST (0) se șterge de pe stivă. (În mnemonica instrucțiunii apare litera P).

Instrucțiuni aritmetice

FADD	$ST(0) \leftarrow ST(0) + ST(1)$
FADD op	$ST(0) \leftarrow ST(0) + "op"$ din memorie sau stivă Operație în virgulă mobilă
FADD op	$ST(0) \leftarrow ST(0) + "op"$ din memorie sau stiva Operație cu date întregi

FADD ST (i), ST (0) $ST(i) \leftarrow ST(i) + ST(0)$;ST (0) se elimină

FSUB $ST(0) \leftarrow ST(0) - ST(1)$

FSUB op $ST(0) \leftarrow ST(0) - \text{"op"}$ din memorie sau stivă

Operație în virgulă mobilă

FISUB op $ST(0) \leftarrow ST(0) - \text{"op"}$ din memorie sau stivă

Operație cu date întregi

FSUB ST (i), ST (0) $ST(i) \leftarrow ST(i) - ST(0)$;ST (0) se elimină

FSUBR ST (i) $ST(i) \leftarrow ST(i) - ST(0)$;Este opusul instrucțiunii FSUB ST (i)

FMUL $ST(0) \leftarrow ST(0) * ST(1)$

FMUL op $ST(0) \leftarrow ST(0) * \text{"op"}$ din memorie sau stivă.

Operație în virgulă mobilă

FIMUL op $ST(0) \leftarrow ST(0) * \text{"op"}$ din memorie sau stivă

Operație cu date întregi

FMULP ST (i), ST (0) $ST(i) \leftarrow ST(i) * ST(0)$;ST (0) se elimină

FDIV $ST(0) \leftarrow ST(1) / ST(0)$

FDIV op $ST(0) \leftarrow ST(0) / \text{"op"}$ din memorie sau stivă

Operație în virgulă mobilă

FDIV op $ST(0) \leftarrow ST(0) / \text{"op"}$ din memorie sau stivă

Operație cu date întregi

FDIVP ST (i), ST (0) $ST(i) \leftarrow ST(i) / ST(0)$;ST (0) se elimină

FDIVR ST (i) $ST(i) \leftarrow ST(i) / ST(0)$;Este opusul instrucțiunii FDIV ST (i).

Instrucțiuni pentru compararea valorilor numerice

FCOM Se compară valorile din ST (0) și ST (1) poziționând indicatorii C3, C2 și C0

FCOM op Se compară valorile din ST (0) și din memorie sau stivă (variabilă în virgulă mobilă) poziționând indicatorii C3, C2 și C0.

FICOM op Se compară valorile din ST (0) și din memorie sau stivă (variabilă întreagă) poziționând indicatorii C3, C2 și C0.

FCOMP Se compară valorile din ST (0) și ST (1), poziționând indicatorii C3, C2 și C0. Se elimină ST (0) de pe stivă.

FICOMP op Se compară valorile din ST (0) și din memorie sau stivă (variabilă întreagă) poziționând indicatorii C3, C2 și C0. ST (0) se elimină de pe stivă.

FCOMPP Se compară valorile din ST (0) și ST (1), poziționând indicatorii C3, C2 și C0. Se elimină ST (0) și ST (1) de pe stivă.

FTST Poziționează indicatorii C3, C2 și C0 în funcție de rezultatul comparării valorii din ST (0) cu zero.

FXAM Poziționează biții de condiție în funcție de valoarea din ST (0). Nu se face comparație!

Observații:

- FCOMP și FCOMPP oferă posibilitatea cea mai facilă de a elimina una sau două elemente de pe stivă

- FXAM se folosește pentru analiza condițiilor mai deosebite de erori de calcul.

8.3.5 Funcții în virgulă mobilă

FSQRT Radical –rădăcina pătrată a numărului din ST (0) se scrie în ST (0). Numărul trebuie să fie pozitiv, altfel rezultatul nu are sens.

FSCALE Puterea lui 2. Scrie în ST (0) valoarea lui ST (0) înmulțită cu 2 la puterea ST (1), adică $ST(0) \leftarrow ST(0) * (2^{ST(1)})$. ST (1) trebuie să fie număr întreg, iar valoarea absolută din ST (0) să fie mai mic de 2^{15} .

FPREM Calculul restului parțial. ST (0) se împarte cu ST (1) și pune în ST (0) $\leftarrow ST(0) - ST(1) * (\text{întregul mai mic cel mai apropiat de } ST(0) / ST(1))$.

FRMDINT Rotunjire. În locul lui ST (0) se pune ST (0) rotunjit. Strategia de rotunjire se stabilește în cuvântul de comandă.

FXTRACT Valoarea din ST (0) se desface în caracteristică (în ST (0)) și mantisa (în ST (1)).

FABS În locul lui ST (0) se pune valoarea lui absolută.

FCHS Schimbarea semnului ST (0).

FPTAN Tangenta parțială. Tangenta unghiului din ST (0) este calculată, iar pe stiva se pune pe lângă aceasta, valoarea 1.

FPATAN Arctangentă parțială. În ST (0) se va încărca arctangentă valorii ST (1) /ST (0). Inițial ST (0) trebuie să fie pozitiv, iar ST (1) mai mare ca ST (0).

F2XM1 Calculul puterii lui 2. În locul lui ST (0) se va scrie $2^{ST(0)-1}$. Inițial ST (0) trebuie să fie cuprins între 0 și 0.5.

FYL2X Logaritmare. $ST(0) \leftarrow ST(1) * \text{LOG}_2(ST(0))$. ST (0) trebuie să fie pozitiv, iar ST (1) orice număr finit.

FYL2XP1 Logaritmare. $ST(0) \leftarrow ST(1) * \text{LOG}_2(ST(0) + 1)$. ST (0) trebuie să fie pozitiv mai mic decât 0.3, iar ST (1) orice număr finit.

Observații:

- Cu ajutorul tangentei se pot calcula sin și cos
- Cu F2XM1 se poate calcula orice exponent
- Pentru calculul de exponent ST (0) la puterea ST (1) se recomandă aplicarea succesivă a funcțiilor FYL2X apoi F2XM1!

8.3.6 Instrucțiuni de comandă

Instrucțiunile de comandă au ca sarcină coordonarea acțiunilor coprocesorului. De obicei nu au o semnificație aritmetică, dar există câteva care influențează serios acțiunile aritmetice ale coprocesorului, deoarece ele salvează sau încarcă starea coprocesorului, adică toate registrele de lucru. În aceste registre este inclusă și stiva, deci aceste instrucțiuni pot fi privite ca instrucțiuni gigantice de scriere și salvare.

FINIT Inițializare-aducerea coprocesorului într-o stare inițială cunoscută; "software reset". După efectuarea instrucțiunii FINIT toate registrele coprocesorului se vor afla în starea inițială iar stiva va fi goală.

FENI Acceptarea întreruperilor-pentru ca coprocesorul să genereze o întrerupere la apariția unei erori, pe lângă poziționarea biților corespunzători registrului de comandă este nevoie de acceptarea explicită a întreruperilor.

FDISI Ignorarea întreruperilor-această instrucțiune ignoră întreruperile indiferent de starea biților corespunzători ai registrului de comandă; pentru acceptarea unor noi întreruperi trebuie să avem o nouă instrucțiune FENI

FLDCW adr Încărcarea din memorie a registrului de comandă se încarcă în registrul de comandă cuvântul de la adresa “adr” de memorie

FSTCW adr Salvarea registrului de comandă în memorie-salvarea registrului de comandă în variabilă pe un cuvânt aflat în memorie.

FSTSW adr Salvarea registrului de stare-salvarea registrului de stare într-un cuvânt de memorie aflat la adresa “adr” de memorie.

FCLEX Ștergerea biților de definire a excepțiilor –instrucțiunea șterge biții respectivi indiferent de starea biților de eroare.

FSTENV adr Salvarea mediului-se salvează registrele interne ale coprocesorului într-o zonă de memorie începând de la adresa “adr” și având o lungime de 14 octeți.

FLDENV adr Încărcarea mediului-se încarcă din memorie de la adresa “adr” o zonă de 14 octeți în registrele interioare ale coprocesorului.

FSAVE adr Salvarea stării-salvarea stării coprocesorului (registrele interioare și stiva) în zona de memorie care începe la adresa “adr” și are o lungime de 94 octeți.

FRSTOR adr Încărcarea stării –încărcarea stării coprocesorului (registrele interioare și stiva) din zona de memorie care începe la adresa “adr” și are o lungime de 94 octeți.

FINCSTP Incrementarea indicatorului de stivă-după acțiunea instrucțiunii se incrementează cu un indicator de stivă; elementul care a devenit astfel ST (0) rămâne neschimbat (fapt semnalat de biții corespunzători ai registrului de descriere a stivei).

FDECSTP Decrementarea indicatorilor de stivă-după executarea instrucțiunii indicatorul de stivă e decrementat cu unu; elementele stivei rămânând neschimbate (fapt semnalat de biții corespunzători ai registrului de descriere a stivei).

FFREE ST (i) Ștergerea elementului “i” din stivă; operația nu afectează indicatorul de stivă.

FNOP Nici o operație.

FWAIT Așteptare terminare operație curentă (analog cu instrucțiunea WAIT pentru 8086).

8.4 Mersul lucrării

1. Discuții legate de problemele întâmpinate în înțelegerea documentației scrise.
2. Se va studia exemplul s8ex1.asm, care calculează aria unui cerc și volumul unei sfere, dându-se raza acestora. Să se traseze programul, utilizând Olly Debugger, urmărind în special stiva coprocesorului. De ce apar instrucțiuni în plus, față de codul scris?
3. Să se scrie un program care calculează sinusul unui unghi. Indicație: $\tan(x) = \sin(x)/\cos(x)$; $\sin^2(x) + \cos^2(x) = 1$.
4. Se va scrie un program pentru calcularea radicalului de ordin 3 din 2. Indicație: se vor folosi instrucțiunile F2XM1 și FYL2X.